SUPERMART ANALYTICS

DATA ANALYTICS CAPSTONE PROJECT

PostgreSQL · Intermediate Level

Dataset Scale	Period Covered	Currency
800 customers · 35 employees · 68 products · 2,000 orders · 5,508 order items	January 2021 – June 2024	Nigerian Naira (₦)

📌 Project **Context**

SuperMart is a Nigerian retail chain operating across six regions (North, South, East, West, North-Central, South-South), serving customers in 30 cities. The company sells 68 products across 8 categories and is staffed by 35 employees organised into three tiers: Regional Managers, Sales Managers, and Sales Reps.

As a Data Analyst at SuperMart, you have been tasked with querying the sales database to generate insights on revenue performance, employee effectiveness, customer behaviour, and inventory.

Database Schema

Table	Key Columns
regions	region_id, region_name
categories	category_id, category_name
employees	employee_id, first_name, last_name, role, region_id, hire_date, salary
customers	customer_id, first_name, last_name, email, city, country, registration_date
products	product_id, product_name, category_id, unit_price, stock_quantity
orders	order_id, customer_id, employee_id, order_date, status, shipping_city
order_items	order_item_id, order_id, product_id, quantity, unit_price, discount

Table Relationships

```
regions ──────────< employees
categories ───────< products
customers ────────< orders >─────── employees
orders ──────────────< order_items >── products
```

⚠️ Rules That Apply to Every Question

- **Revenue formula (apply consistently):** quantity * unit_price * (1 - discount / 100.0)
- **Order statuses:** 'Delivered' · 'Pending' · 'Cancelled'
- **Employee roles:** 'Regional Manager' · 'Sales Manager' · 'Sales Rep'
- **Unless a question says otherwise, include ALL order statuses in your results.**
- **All prices are in Nigerian Naira (₦).**

Section A — Fundamentals

Topics: SELECT · WHERE · DISTINCT · ORDER BY · LIMIT

Question 1

a. Retrieve the **first_name**, **last_name**, and **email** of every customer who lives in Lagos. Sort results alphabetically by **last_name**, then by **first_name**.

b. List the names of all distinct cities to which SuperMart has shipped at least one order. Sort alphabetically.

c. Display the top 10 most expensive products by **unit_price**. Show **product_name**, **category_id**, and **unit_price**, ordered from most to least expensive.

d. List all employees hired on or after 1st January 2021. Display their full name (**first_name** and **last_name** concatenated as one column called **full_name**), **role**, **hire_date**, and **salary**, ordered by **hire_date** ascending.

e. Retrieve all orders placed in December across any year. Show **order_id**, **order_date**, **status**, and **shipping_city**. Order by **order_date** descending.

Section B — Aggregate Functions

Topics: COUNT · SUM · AVG · MIN · MAX · ROUND

Question 2

a. How many orders exist for each **status**? Display the status, the count, and each status as a percentage of all orders, rounded to 2 decimal places. Label the percentage column **pct_of_total**. Order by count descending.

b. For each product category, calculate the **minimum**, **maximum**, and **average unit_price** of products in that category. Round the average to 2 decimal places. Display **category_name** (not just the ID). Order by average price descending.

You will need to join products with categories.

c. Across all rows in **order_items**, calculate: the total revenue generated, the average revenue per line item, the maximum revenue from a single line item, and the minimum revenue from a single line item. Round all values to 2 decimal places and label each column clearly.

d. How many distinct customers have placed at least one order? What is the average number of orders per ordering customer, rounded to 2 decimal places? Display both figures as separate columns in a single result row.

Section C — Grouping

Topics: GROUP BY · HAVING

Question 3

a. Count the number of customers who registered each year between 2018 and 2024. Display the registration year and the count. Order by year ascending.

b. Which shipping cities received more than 10 delivered orders in total? Display the city name and the count of delivered orders, ordered by count descending.

c. Find all products whose total quantity sold across all **order_items** exceeds 50 units. Display **product_id**, **product_name**, and total quantity sold. Order by total quantity descending.

💡 Do not filter by order status — include all orders.

d. Show each employee's full name and the total number of orders they handled. Return only employees who handled 20 or more orders. Order by order count descending.

💡 Think carefully about whether employees with zero orders should appear in this result — read the question closely.

e. For each year in the dataset (2021–2024), show the total number of orders placed and the count of distinct customers who ordered that year. Order by year ascending.

Section D — LIKE & ILIKE

Topics: LIKE · ILIKE · % wildcard · _ wildcard

Question 4

a. SuperMart wants to run a Gmail campaign. Retrieve the **first_name**, **last_name**, and **email** of all customers whose email address ends with **@gmail.com**. Order alphabetically by **last_name**.

b. A product manager needs a list of all products whose names include the word "set" anywhere, regardless of case. Use **ILIKE**. Display **product_name**, **category_id**, and **unit_price**, ordered by **unit_price** descending.

c. Find all customers whose last name begins with the letters 'Ad' (case-insensitive). Display full name, city, and **registration_date**.

d. Retrieve all products whose names contain "combo", "kit", or "pack" anywhere in the name (case-insensitive). Use **ILIKE** with **OR**. Display **product_name**, **category_id**, and **unit_price**.

e. Find all customers whose city name contains the letter sequence 'an' (case-insensitive — e.g. Kano, Kaduna). Display **first_name**, **last_name**, and **city**. Order by **city**, then **last_name**.

Section E — JOINs

Topics: INNER JOIN · LEFT JOIN · Multi-table JOIN

Question 5

a. Display the 50 most recent orders. For each, show: **order_id**, the customer's full name, the handling employee's full name, **order_date**, **status**, and **shipping_city**. Use INNER JOINs. Order by **order_date** descending.

b. List all 800 customers alongside the total number of orders they have placed. Customers who have never ordered should show 0. Display **customer_id**, full name, **city**, and **order_count**. Order by **order_count** descending, then **last_name** ascending.

c. Produce a detailed order line report containing every row in **order_items**. For each row show: **order_id**, **order_date**, customer full name, **product_name**, **quantity**, **unit_price**, **discount**, and a calculated column **line_total** using the revenue formula. Order by **order_id** ascending, then **product_name** ascending.

d. Show all 35 employees with their **full_name**, **role**, **region_name** (from the **regions** table), and the total number of orders they have handled. Include employees with zero orders (show 0). Order by total orders descending, then **last_name** ascending.

e. For each product category, list every product alongside the total number of distinct orders it has appeared in and the total quantity sold. Display **category_name**, **product_name**, **times_ordered**, and **total_qty_sold**. Order by **category_name**, then **total_qty_sold** descending.

Section F — CASE Expressions

Topics: CASE WHEN · CASE inside aggregates

Question 6

a. Assign a price tier label to every product using the table below. Display **product_name**, **category_name** (joined from **categories**), **unit_price**, and **price_tier**. Order by **unit_price** ascending.

Condition	Label
unit_price < 10,000	'Budget'
unit_price between 10,000 and 99,999	'Mid-Range'
unit_price >= 100,000	'Premium'

b. Classify each of the 35 employees into a pay band based on their **salary**. Display **full_name**, **role**, **salary**, and **pay_band**. Order by **salary** descending.

Condition	Label
salary >= 100,000	'Executive'
salary between 80,000 – 99,999	'Senior'
salary < 80,000	'Entry Level'

c. For each order, calculate the total order value (sum of all its line totals), then classify it using the table below. Display **order_id**, **order_date**, **status**, **total_order_value** (rounded to 2 dp), and **value_category**. Order by **total_order_value** descending.

Hint: JOIN orders with order_items, GROUP BY order, then apply CASE to the aggregated total.

Condition	Label
Total > 500,000	'High Value'
Total between 100,000 and 500,000	'Medium Value'
Total < 100,000	'Low Value'

d. Using a single query with CASE inside an aggregate, count how many products in each category fall into each price tier. Display one row per category with columns: **category_name**, **budget_count**, **mid_range_count**, **premium_count**.

Hint: Use COUNT(CASE WHEN ... THEN 1 END) for each tier.

Section G — Subqueries

Topics: Scalar · IN / NOT IN · NOT EXISTS · Derived tables

Question 7

a. Find all products whose **unit_price** is above the average unit price of all products in the catalogue. Display **product_name**, **category_id**, and **unit_price**. Order by **unit_price** descending.

Use a scalar subquery in the WHERE clause. Do not use a CTE.

b. List all customers who have placed at least one order. Display their full name and **city**. Solve this using a subquery with **IN** — do not use a JOIN.

c. Find all products that have never appeared in any order. Display **product_id**, **product_name**, **category_id**, and **unit_price**.

Use either NOT IN or NOT EXISTS.

d. Using subqueries, find the top 5 customers by total lifetime revenue (all statuses, all order items). Display their full name, **city**, and total lifetime revenue rounded to 2 decimal places.

💡 Do not use a CTE. Use a subquery in the FROM clause (a derived table).

e. Find all customers whose total lifetime revenue exceeds the average lifetime revenue across all ordering customers. Display their full name, **city**, and total revenue (rounded to 2 dp). Order by total revenue descending.

💡 You will need a subquery in FROM (to calculate per-customer revenue) and in HAVING or WHERE (to compare against the average). Do not use a CTE.

Section H — CTEs (Common Table Expressions)

Topics: WITH clause · Single CTE · Chained CTEs · CTE + CASE

Question 8

a. Using a single CTE, calculate the total revenue per customer across all their orders (all statuses). From the outer query, return only the top 10 customers by revenue. Display **customer_id**, full name, **city**, and **total_revenue** rounded to 2 dp.

b. Using a CTE, identify the single best-selling product (by total quantity sold) in each category. Display **category_name**, **product_name**, and **total_qty_sold**.

💡 Hint: Calculate total qty per product in the CTE, then in the outer query filter to the product with the maximum qty within each category.

c. Using two chained CTEs, analyse monthly performance for the year 2023 only:

- CTE 1: Total revenue per calendar month in 2023 (all statuses).
- CTE 2: The average monthly revenue across all months of 2023.
- Final query: Each month number, total revenue (rounded to 2 dp), and a column called **vs_average** — set to **'Above Average'** if that month beat the average, otherwise **'Below Average'**. Order by month ascending.

d. Using CTEs, produce a customer frequency segmentation report. Calculate how many total orders each customer has placed, then classify each customer using the table below. Return one row per segment showing the **segment** label and **customer_count**. Order by **customer_count** descending.

💡 Hint: CTE 1 — count orders per customer (LEFT JOIN to include zero-order customers). Outer query — apply CASE, then GROUP BY.

Orders Placed	Segment
8 or more	'High Frequency'
4 to 7	'Regular'
1 to 3	'Occasional'
0	'Inactive'

e. Using a CTE, compute the year-over-year total revenue from delivered orders for each year in the dataset (2021, 2022, 2023, and the first half of 2024). Display **order_year** and **total_revenue** (rounded to 2 dp). Order by year ascending.

Section I — Capstone Challenge

Topics: CTEs + JOINs + GROUP BY + Aggregates + CASE — Multi-concept

Question 9 — Employee Sales Performance Report

SuperMart's board has requested a comprehensive Employee Sales Performance Dashboard for all delivered orders placed between 1 January 2021 and 30 June 2024.

Write a single SQL query — using at least two CTEs — that returns the following for every employee:

Output Column	Description
employee_name	Full name (first + last)
role	Employee's role
region_name	Region the employee belongs to
total_delivered_orders	Count of orders with status = 'Delivered'
total_revenue	Sum of revenue from all delivered order items (rounded to 2 dp)
avg_order_value	Average revenue per delivered order (rounded to 2 dp)
best_single_order	Revenue of the single highest-value delivered order (rounded to 2 dp)
performance_band	Classification — see table below

Performance Band Classification:

Condition	Label
total_revenue > 5,000,000	'Elite'
total_revenue between 1,000,000 and 5,000,000	'Strong'
total_revenue between 100,000 and 999,999	'Developing'
total_revenue < 100,000 OR no delivered orders	'Inactive'

Requirements:

- Include all 35 employees, even those who handled zero delivered orders.
- Employees with no delivered orders must show **0** for all numeric columns.
- Order results by **total_revenue** descending, then **employee_name** ascending.

Suggested CTE structure:

- CTE 1 — Calculate each order's total revenue by summing its line items from **order_items**.
- CTE 2 — Join CTE 1 with **orders** (filter **status = 'Delivered'**) and aggregate per **employee_id**: total orders, total revenue, and best single order.
- Final SELECT — LEFT JOIN CTE 2 onto **employees**, then join **regions** for the region name. Apply **COALESCE** for zero-order employees and **CASE** for the performance band.

Section J — Bonus Challenge — Extension for Fast Finishers

Topics: All concepts combined

Question 10 — Customer Lifetime Value Report

Management also wants to understand customer purchasing behaviour over the lifetime of the business. Write a query that produces the following for every customer registered before 2024:

Output Column	Description
customer_name	Full name
city	Customer's city
registration_year	Year extracted from registration_date
total_orders	All orders regardless of status
delivered_orders	Orders with status = 'Delivered' only
cancelled_orders	Orders with status = 'Cancelled' only
lifetime_revenue	Total revenue from delivered orders (rounded to 2 dp)
avg_order_value	Average revenue per delivered order (rounded to 2 dp); show 0 if none
customer_segment	Classification — see table below

Customer Segment Classification:

Condition	Label
lifetime_revenue > 500,000 AND delivered_orders >= 5	'VIP'
lifetime_revenue between 100,000–500,000 OR delivered_orders between 2–4	'Loyal'
delivered_orders = 1	'One-Time Buyer'
0 delivered orders but at least 1 total order	'No Conversions'
0 total orders	'Inactive'

Requirements:

- Use at least one CTE.
- Include customers with zero orders — show **0** for numeric columns.
- Order by **lifetime_revenue** descending, then **customer_name** ascending.

📋 Quick Reference

- Revenue formula: **quantity * unit_price * (1 - discount / 100.0)**
- Date range: **2021-01-01** to **2024-06-30**
- Order statuses: **'Delivered'** · **'Pending'** · **'Cancelled'**
- Employee roles: **'Regional Manager'** · **'Sales Manager'** · **'Sales Rep'**